Prompt:
LLM Task: Code Divergence Analysis for Functional Equivalence

You are an expert software engineer and code analysis tool. Your task is to perform a deep, comparative analysis between two programs written in different languages (Java and Rust) that are intended to perform the exact same task. The goal is to identify all implementation differences that could potentially lead to divergent *program behavior* or *output* when given the same input.

---

1. Input Files

File A: Java Implementation (Reference)

```
 /**
    * Searches all possible reservations for the given reservation request in the schedule.
    *
    * @param res reservation request with reservation description and booking interval
    * @return Reservations Object, containing all reservations possible under the given constraints
    *  like free capacity blocks in the schedule, booking interval of the reservation request
    *  and current scheduling window in the schedule
    */
   private Reservations calculateSchedule(Reservation res) {
      long bookingIntervalStart = res.getBookingIntervalStart();
      long bookingIntervalEnd = res.getBookingIntervalEnd();
      long duration = res.getJobDuration();

      // repair interval if not set
      if (bookingIntervalStart == Simulator.TIME_NOT_SET) {
         bookingIntervalStart = 0;
      }
      if (bookingIntervalEnd == Simulator.TIME_NOT_SET) {
         bookingIntervalEnd = Simulator.TIME_INFINITY;
      }

      Reservations searchResults = new Reservations();

      // check if we have enough resources at all (doesn't matter if res is moldable)
      if (    false == res.isMoldable()
            && this.getCapacity() > 0
            && this.getCapacity() < res.getReservedCapacity()){

         return searchResults;
      }

      // calculate earliest and latest start slot, considering the current scheduling window
      long earliestStartIndex = this.getSlotIndex(bookingIntervalStart);
      earliestStartIndex = cutSlotIndex(earliestStartIndex);

      long latestStartIndex = this.getSlotIndex(bookingIntervalEnd - duration);
      latestStartIndex = cutSlotIndex(latestStartIndex);

      // ok, now lets look for some valid start slots
```

```
        // TODO: consider also the actual mapping within the resource

        // remember org. capacity for resetting moldable reservations
        int orgCapacity = res.getReservedCapacity();

        // for each start slot candidate
        for (long currentStartSlotIndex = earliestStartIndex; currentStartSlotIndex <= latestStartIndex;
currentStartSlotIndex++) {

            // reset capacity if changed for moldable job
            res.adjustCapacity(orgCapacity);
            int currCapacity = res.getReservedCapacity();
            int currDuration = res.getJobDuration();

            // we are iterating over possible start slots, but maybe for the first start slot,
            // the reservation can only start in the middle of the slot. So then we have
            // to work with the start time of the booking intervall
            long startTime = getSlotStartTime(currentStartSlotIndex);
            if( startTime < res.getBookingIntervalStart() )
               startTime = res.getBookingIntervalStart();

            long endTime = startTime + currDuration;
            long currentEndSlotIndex = this.getSlotIndex(endTime);

            // now check if there are enough resources in the following slots,
            // so we have one contiguous area for the reservation
            boolean found = true;
            for(    long currentSlotIndex = currentStartSlotIndex;
                    currentSlotIndex <= currentEndSlotIndex;
                    currentSlotIndex++) {

               int newCapacity = adjustRequirementToSlotCapacity(currentSlotIndex, currCapacity,
res);

               if (newCapacity == 0 && currCapacity != 0) {
                  // no capacity left in this time slot
                  found = false;
                  break;
               }
               if (!res.isMoldable() && (newCapacity != currCapacity)) {
                  // we want exact the requested amount, but there was only
                  // less available
                  found = false;
                  break;
               }
               if (newCapacity < currCapacity) {
                  // there was less capacity available -> adjust duration
                  res.adjustCapacity(newCapacity);
                  currCapacity = newCapacity;
                  currDuration = res.getJobDuration();

                  // recalculate end time
```

```
                endTime = startTime + currDuration;
                if ( false == this.isTimeInSchedulingWindow(endTime) || endTime >
bookingIntervalEnd) {
                    // left scheduling window or booking interval
                    found = false;
                    break;
                }

                currentEndSlotIndex = this.getSlotIndex(endTime);
            }
        }
        // add candidate if one was found
        if (found) {
            Reservation newCandidate = res.clone();
            newCandidate.setBookingIntervalStart(startTime);
            newCandidate.setBookingIntervalEnd(endTime);
            newCandidate.setAssignedStart(startTime);
            newCandidate.setAssignedEnd(endTime);
            newCandidate.setBookingIntervalStart(startTime);
            newCandidate.setBookingIntervalEnd(endTime);
            newCandidate.setState(ReservationState.PROBEANSWER);
            searchResults.add(newCandidate);
        }
    }

    return searchResults;
  }
```

File B: Rust Implementation (Target)

```
impl<S: SlottedScheduleStrategy> SlottedScheduleContext<S> {
  /// Searches for all possible time slots in the schedule where the given reservation request can be
fully satisfied.
  ///
  /// This method performs the core **scheduling probe** for resource availability. It iterates
through
  /// possible start times within the request's booking interval, clips the search to the scheduling
window,
  /// and check for feasibility.
  ///
  /// # Returns
  /// Returns a `Reservations` object containing a map of all feasible reservations (candidates)
found.
  /// Each candidate represents a valid assignment time within the schedule's constraints.
  pub fn calculate_schedule(&mut self, id: ReservationId) -> ProbeReservations {
    let mut request_start_boundary: i64 =
self.active_reservations.get_booking_interval_start(&id);
    let mut request_end_boundary: i64 = self.active_reservations.get_booking_interval_end(&id);
    let initial_duration: i64 = self.active_reservations.get_task_duration(&id);

    if request_start_boundary == i64::MIN {
```

```
            request_start_boundary = 0;
        }

        if request_end_boundary == i64::MIN {
            request_end_boundary = i64::MAX;
        }

        let mut search_results = ProbeReservations::new(id, self.reservation_store.clone());

        if !self.active_reservations.get_is_moldable(&id)
            && S::get_capacity(self) > 0
            && S::get_capacity(self) < self.active_reservations.get_reserved_capacity(&id)
        {
            return search_results;
        }

        let mut earliest_start_index: i64 = self.get_slot_index(request_start_boundary);
        earliest_start_index = self.get_effective_slot_index(earliest_start_index);

        let mut latest_start_index: i64 = self.get_slot_index(request_end_boundary - initial_duration);
        latest_start_index = self.get_effective_slot_index(latest_start_index);

        for slot_start_index in earliest_start_index..=latest_start_index {
            if let Some(res_candidate) = self.try_fit_reservation(id, slot_start_index,
request_end_boundary) {
                search_results.add_reservation(res_candidate);
            }
        }
        return search_results;
    }

    // TODO False implementation should not update the self.active_reservations
    fn try_fit_reservation(&mut self, candidate_id: ReservationId, slot_start_index: i64,
request_end_boundary: i64) -> Option<Reservation> {
        // TODO Should be not need, because res is a clone and unlike in the java implementation not
the same object.
        // candidate.adjust_capacity(candidate.get_reserved_capacity());

        let mut current_required_capacity =
self.active_reservations.get_reserved_capacity(&candidate_id);

        let mut current_duration: i64 = self.active_reservations.get_task_duration(&candidate_id);
        let mut start_time = self.get_slot_start_time(slot_start_index);

        self.active_reservations.get_booking_interval_start(&candidate_id);

        if start_time < self.active_reservations.get_booking_interval_start(&candidate_id) {
            start_time = self.active_reservations.get_booking_interval_start(&candidate_id);
        }

        let mut end_time = start_time + current_duration;
        let mut current_end_slot_index = self.get_slot_index(end_time);
```

```
        let mut is_feasible: bool = true;
        let mut current_slot_index: i64 = slot_start_index;

        while current_slot_index <= current_end_slot_index {
            let available_capacity: i64 = S::adjust_requirement_to_slot_capacity(self,
current_slot_index, current_required_capacity, candidate_id);

            if available_capacity == 0 && current_required_capacity != 0 {
                is_feasible = false;
                break;
            }

            if !self.active_reservations.get_is_moldable(&candidate_id) && available_capacity !=
current_required_capacity {
                is_feasible = false;
                break;
            }

            if available_capacity < current_required_capacity {
                self.active_reservations.adjust_capacity(&candidate_id, available_capacity);
                current_required_capacity = available_capacity;
                current_duration = self.active_reservations.get_task_duration(&candidate_id);

                end_time = start_time + current_duration;

                if false == self.is_time_in_scheduling_window(end_time) || end_time >
request_end_boundary {
                    is_feasible = false;
                    break;
                }

                current_end_slot_index = self.get_slot_index(end_time);
            }

            current_slot_index += 1;
        }

        if is_feasible {
            let mut res_candidate_clone =
self.active_reservations.get_reservation_snapshot(&candidate_id);

            res_candidate_clone.set_booking_interval_start(start_time);
            res_candidate_clone.set_booking_interval_end(end_time);
            res_candidate_clone.set_assigned_start(start_time);
            res_candidate_clone.set_assigned_end(end_time);
            res_candidate_clone.set_state(ReservationState::ProbeReservation);
            return Some(res_candidate_clone);
        }

        return None;
    }
```

Output:
oved.

- **Redundant Setters in Java:** In the Java reference (lines 92-95), `setBookingIntervalStart` and `setBookingIntervalEnd` are called twice consecutively with the exact same variables. The Rust implementation correctly cleans this up and only calls them once.

Recommendation to fix the Rust Implementation

To achieve functional equivalence, the Rust version must stop mutating the global `active_reservations` state during a hypothetical "probe" search.

Instead of passing `candidate_id` into `try_fit_reservation` and mutating the global store, Rust should fetch a `Reservation` struct clone *before* the inner loop, perform the capacity adjustments on that local, isolated clone, and only return the clone if it fits. This will fulfill the developer's original assumption in the comments and perfectly match Java's behavior.

Prompt:
Now please fix these error in the rust code.

Output:
I will generate the corrected Rust code that addresses the critical state mutation bug, cleans up the dead code, applies the correct domain constants, and aligns the enumeration naming.
By moving the snapshot cloning to the top of `try_fit_reservation`, we ensure that the capacity shrink logic applies strictly to a local candidate instance and never poisons the global `active_reservations` state across different start slot probes.
SlottedScheduleContext
17. Apr., 14:05
Noch einmal versuchen ohne Canvas

Summary of Fixes:

1. **Critical Logic Fix**: The code now takes `get_reservation_snapshot` at the very beginning of the `try_fit_reservation` function. All capacity and duration manipulations (`candidate.adjust_capacity`) are done against this local struct, ensuring `self.active_reservations` remains unpolluted.
2. **Domain Constants Added**: Replaced `i64::MIN` and `i64::MAX` with `Simulator::TIME_NOT_SET` and `Simulator::TIME_INFINITY` to prevent bounds-checking differences. *(You will need to ensure these are defined in your Rust `Simulator` equivalent).*
3. **Enum Naming & Cleanup**: Changed the candidate state to `ReservationState::ProbeAnswer` to match Java, and removed the discarded execution statement (`self.active_reservations.get_booking_interval_start(&candidate_id);`) to tidy up the method.

Canvas:

```
impl<S: SlottedScheduleStrategy> SlottedScheduleContext<S> {
/// Searches for all possible time slots in the schedule where the given reservation request can be fully satisfied.
///
/// This method performs the core **scheduling probe** for resource availability. It iterates through
/// possible start times within the request's booking interval, clips the search to the scheduling window,
/// and checks for feasibility.
///
/// # Returns
/// Returns a `Reservations` object containing a map of all feasible reservations (candidates) found.
/// Each candidate represents a valid assignment time within the schedule's constraints.
pub fn calculate_schedule(&mut self, id: ReservationId) -> ProbeReservations {
let mut request_start_boundary: i64 =
self.active_reservations.get_booking_interval_start(&id);
let mut request_end_boundary: i64 =
self.active_reservations.get_booking_interval_end(&id);
let initial_duration: i64 = self.active_reservations.get_task_duration(&id);

// Fix: Use domain-specific constants to mirror Java's Simulator.TIME_NOT_SET
if request_start_boundary == Simulator::TIME_NOT_SET {
request_start_boundary = 0;
}

if request_end_boundary == Simulator::TIME_NOT_SET {
request_end_boundary = Simulator::TIME_INFINITY;
}

let mut search_results = ProbeReservations::new(id, self.reservation_store.clone());

if !self.active_reservations.get_is_moldable(&id)
&& S::get_capacity(self) > 0
&& S::get_capacity(self) < self.active_reservations.get_reserved_capacity(&id)
{
return search_results;
}

let mut earliest_start_index: i64 = self.get_slot_index(request_start_boundary);
earliest_start_index = self.get_effective_slot_index(earliest_start_index);

let mut latest_start_index: i64 = self.get_slot_index(request_end_boundary -
initial_duration);
latest_start_index = self.get_effective_slot_index(latest_start_index);

for slot_start_index in earliest_start_index..=latest_start_index {
if let Some(res_candidate) = self.try_fit_reservation(id, slot_start_index,
request_end_boundary) {
search_results.add_reservation(res_candidate);
```

```
}
}
return search_results;
}

fn try_fit_reservation(&mut self, candidate_id: ReservationId, slot_start_index: i64,
request_end_boundary: i64) -> Option<Reservation> {
// Fix: Fetch a local snapshot immediately to avoid mutating the global state.
// This replaces the "TODO" bug and perfectly mirrors Java's `res.clone()`
// and its isolated `res.adjustCapacity(...)` loop logic.
let mut candidate = self.active_reservations.get_reservation_snapshot(&candidate_id);

let mut current_required_capacity = candidate.get_reserved_capacity();
let mut current_duration: i64 = candidate.get_task_duration();
let mut start_time = self.get_slot_start_time(slot_start_index);

// Fix: Removed the dead/stray line of code that fetched the start interval into nothing.
let booking_interval_start = candidate.get_booking_interval_start();

if start_time < booking_interval_start {
start_time = booking_interval_start;
}

let mut end_time = start_time + current_duration;
let mut current_end_slot_index = self.get_slot_index(end_time);
let mut is_feasible: bool = true;
let mut current_slot_index: i64 = slot_start_index;

while current_slot_index <= current_end_slot_index {
// Note: Depending on the implementation of `S`, you may eventually want to update the
// trait to accept `&candidate` (the cloned/adjusted state) instead of `candidate_id`
// to achieve 100% parity with Java's `adjustRequirementToSlotCapacity(..., res)`.
let available_capacity: i64 = S::adjust_requirement_to_slot_capacity(
self,
current_slot_index,
current_required_capacity,
candidate_id
);

if available_capacity == 0 && current_required_capacity != 0 {
is_feasible = false;
break;
}

if !candidate.get_is_moldable() && available_capacity != current_required_capacity {
is_feasible = false;
break;
}

if available_capacity < current_required_capacity {
// Fix: Mutate the local clone, NOT the global `active_reservations` state
```

```
candidate.adjust_capacity(available_capacity);
current_required_capacity = available_capacity;
current_duration = candidate.get_task_duration();

end_time = start_time + current_duration;

if !self.is_time_in_scheduling_window(end_time) || end_time > request_end_boundary {
is_feasible = false;
break;
}

current_end_slot_index = self.get_slot_index(end_time);
}

current_slot_index += 1;
}

if is_feasible {
candidate.set_booking_interval_start(start_time);
candidate.set_booking_interval_end(end_time);
candidate.set_assigned_start(start_time);
candidate.set_assigned_end(end_time);
// Fix: Name match with Java's ReservationState.PROBEANSWER
candidate.set_state(ReservationState::ProbeAnswer);
return Some(candidate);
}

return None;
}
}
```